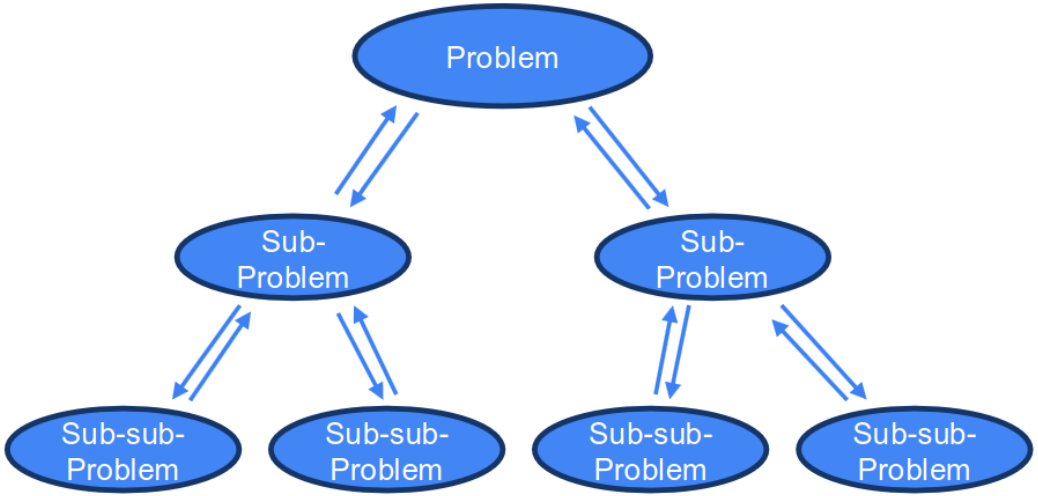
Lecture 20

Divide and Conquer



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

- Project II deliverable 1 due 11/13

Last time

Merge sort

- $\theta(n \log n)$
- $\log n$ many linear time steps
- Derived recurrence relation from pseudocode
- Solved recurrence relation
  - Unrolling
  - Recursion tree

Today

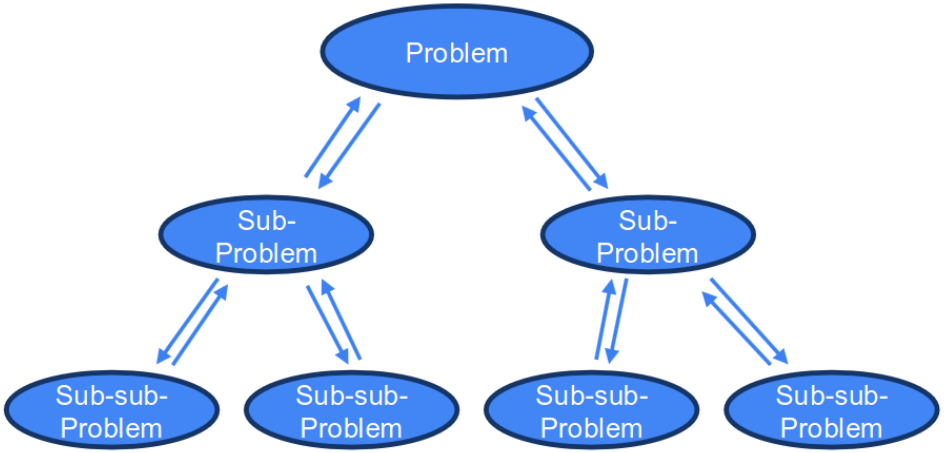
Divide and Conquer algorithms

Nearest neighbors problem

D&C Idea

General Recipe (or pattern)

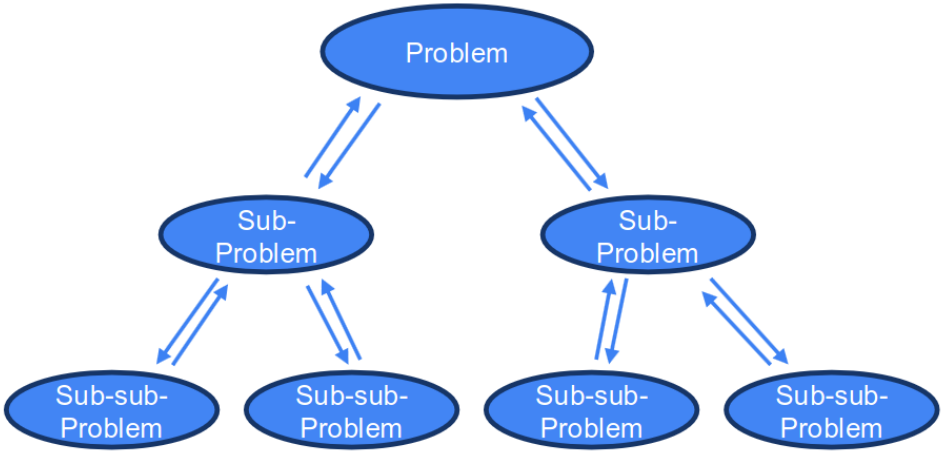
- Divide problem into several parts
- Solve each part recursively
- Combine solutions to sub-problem into overall solution



D&C Idea

General Recipe (or pattern)

- Divide problem into several parts
- Solve each part recursively
- Combine solutions to sub-problem into overall solution

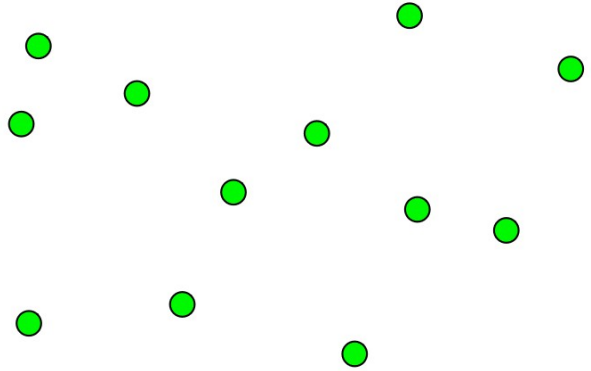


What makes D&C different from dynamic programming?

Nearest Neighbors (2D)

Input: a collection of n points in the plane

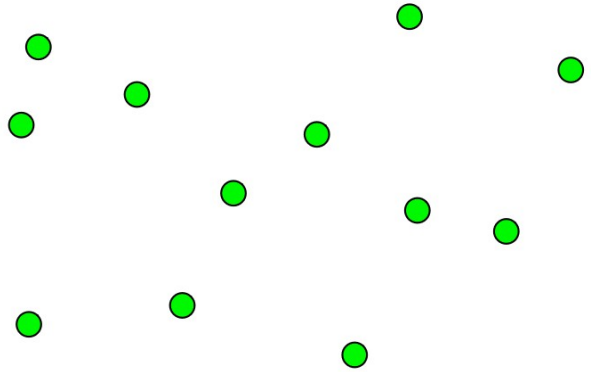
Goal: identify a pair that has the smallest distance between them



Nearest Neighbors (2D)

Input: a collection of n points in the plane

Goal: identify a pair that has the smallest distance between them



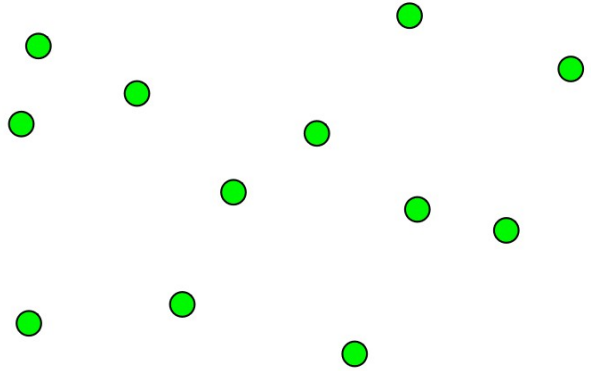
Baseline: check every pair

- $\theta(n^2)$ – how many pairs are there?

Nearest Neighbors (2D)

Input: a collection of n points in the plane

Goal: identify a pair that has the smallest distance between them



Baseline: check every pair

- $\theta(n^2)$ – how many pairs are there?

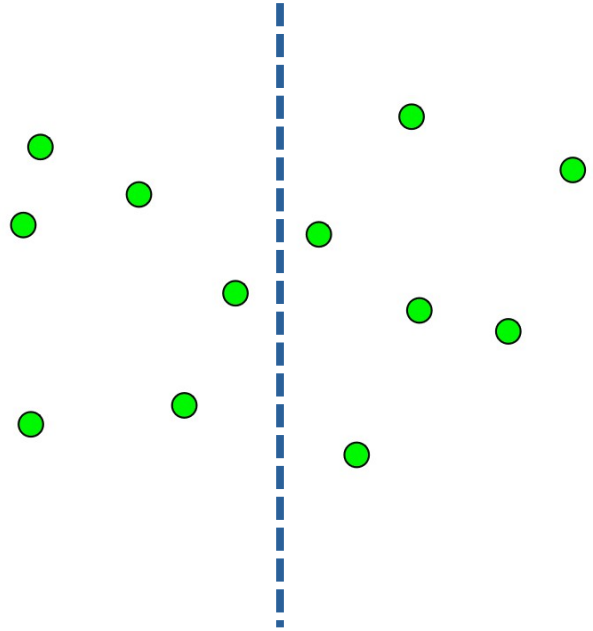
Can we go faster?

- Let's think of a D&C approach

Divide

Strategy

- Split the points in half
- Recursively find the pair of points closest in each half

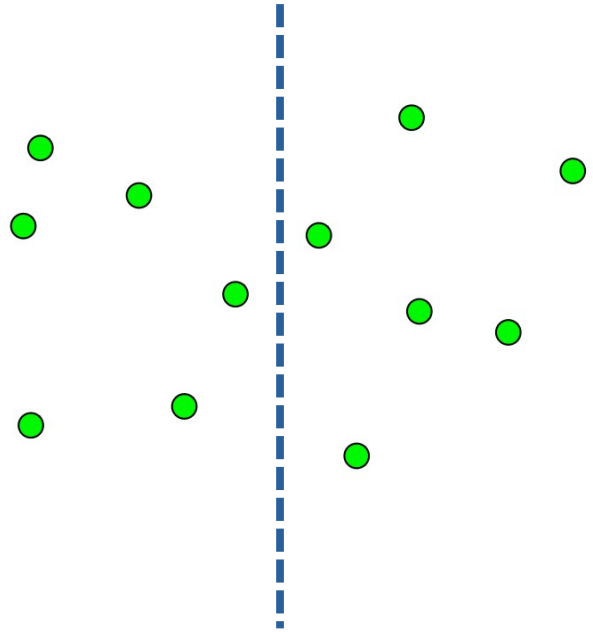


Divide

Strategy

- Split the points in half
- Recursively find the pair of points closest in each half

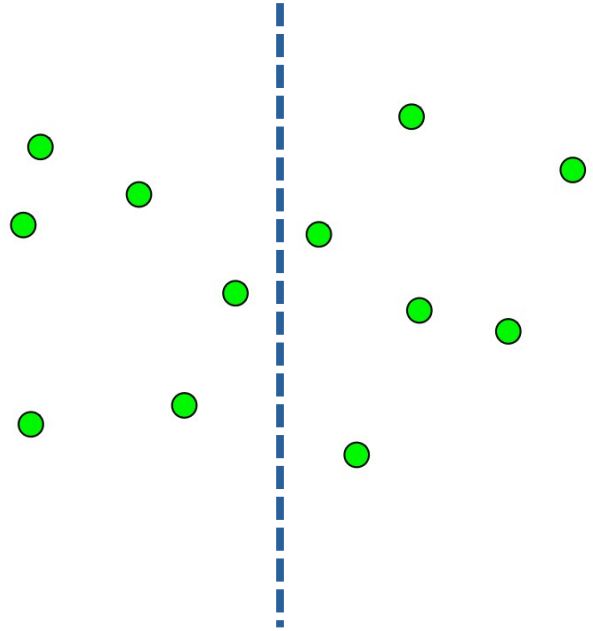
What should be the base case?



Divide

Strategy

- Split the points in half
- Recursively find the pair of points closest in each half



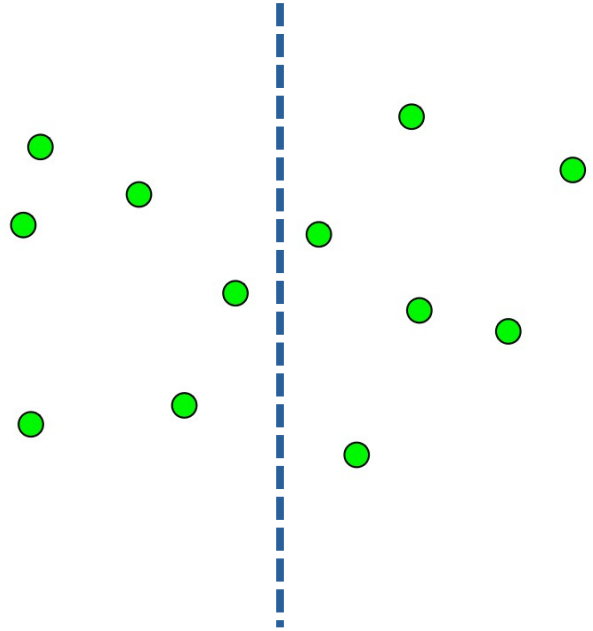
What should be the base case?

- Either two or three points
- We can just check by brute force
- Running time:

Divide

Strategy

- Split the points in half
- Recursively find the pair of points closest in each half



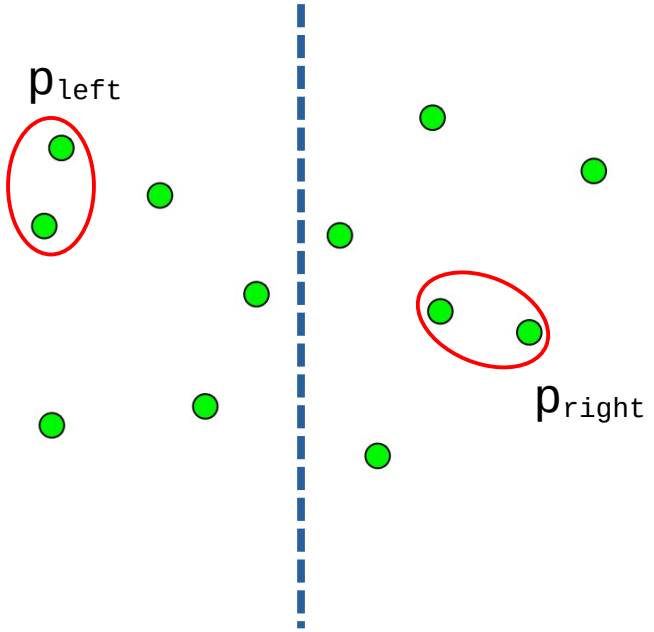
What should be the base case?

- Either two or three points
- We can just check by brute force
- Running time: $O(1)$

Combining

Suppose we've solved our recursive cases

- p_{left} is the closest pair on left with dist d_{left}
- p_{right} is the closest pair on right with dist d_{right}



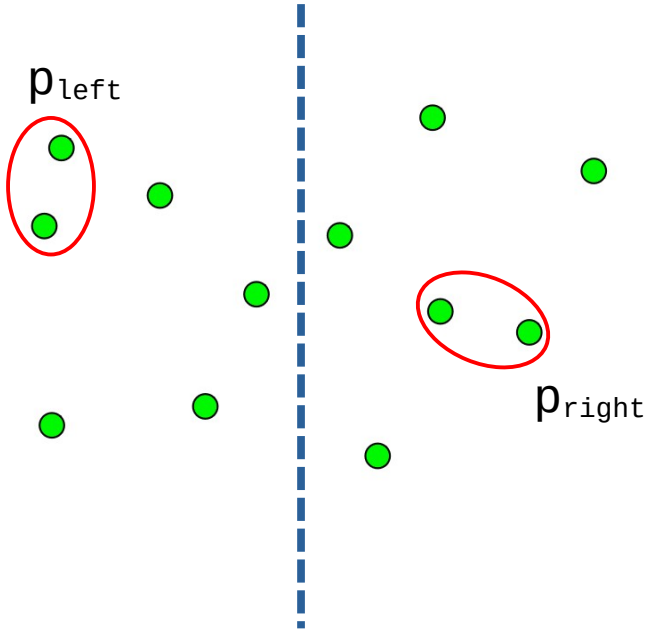
Combining

Suppose we've solved our recursive cases

- p_{left} is the closest pair on left with dist d_{left}
- p_{right} is the closest pair on right with dist d_{right}

Let $d = \min\{d_{left}, d_{right}\}$

- Is d the minimum distance?



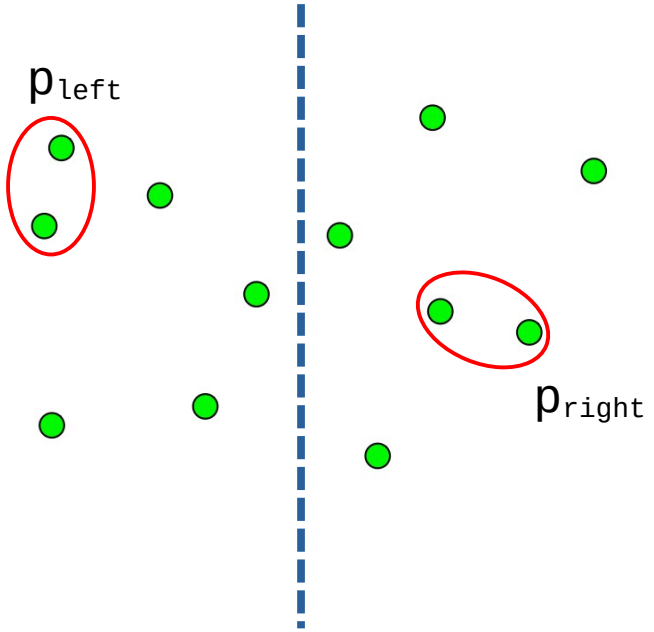
Combining

Suppose we've solved our recursive cases

- p_{left} is the closest pair on left with dist d_{left}
- p_{right} is the closest pair on right with dist d_{right}

Let $d = \min\{d_{left}, d_{right}\}$

- Is d the minimum distance? Not necessarily
- Maybe the minimum distance pair crosses the split!



Combining

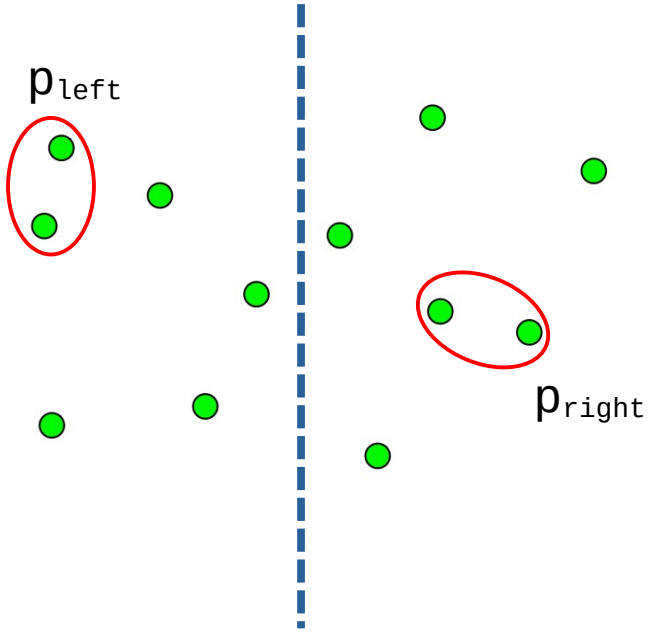
Suppose we've solved our recursive cases

- p_{left} is the closest pair on left with dist d_{left}
- p_{right} is the closest pair on right with dist d_{right}

Let $d = \min\{d_{left}, d_{right}\}$

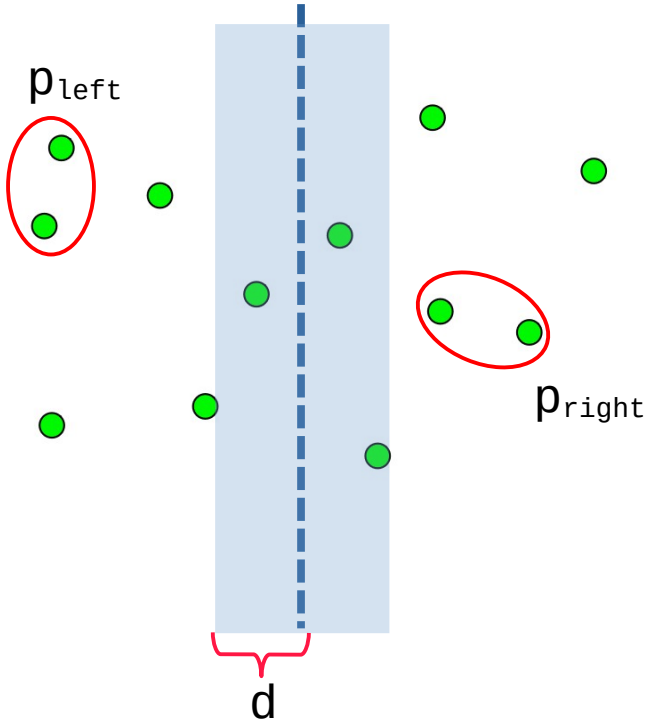
- Is d the minimum distance? Not necessarily
- Maybe the minimum distance pair crosses the split!

Do we have to check every pair that crosses?



Region Near Split

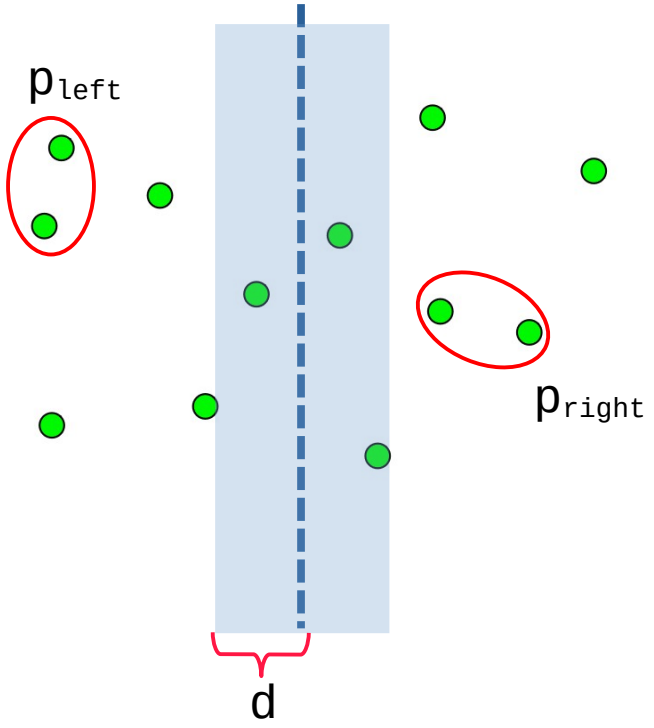
If there is a pair with distance less than d that crosses the split, then it must belong inside the blue box



Region Near Split

If there is a pair with distance less than d that crosses the split, then it must belong inside the blue box

- Otherwise, the pair could not be close enough!



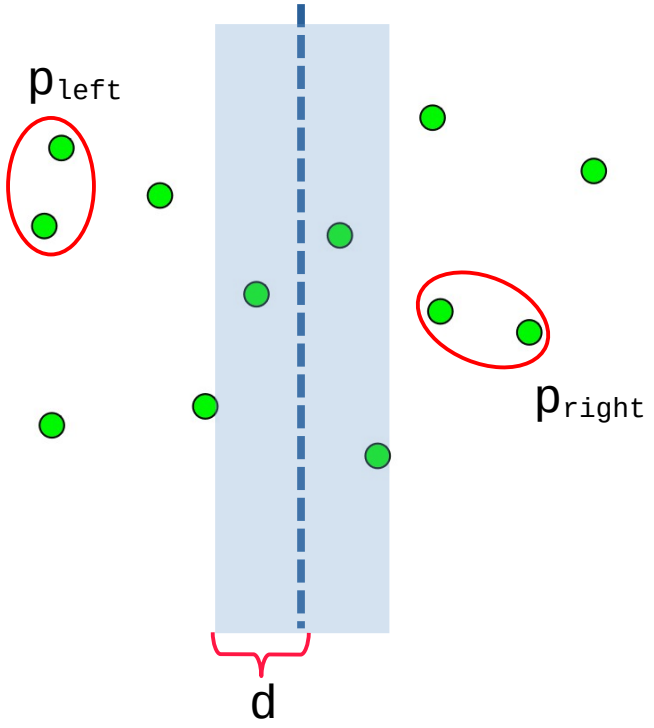
Region Near Split

If there is a pair with distance less than d that crosses the split, then it must belong inside the blue box

- Otherwise, the pair could not be close enough!

Let S_y be a list of the points in the blue box sorted decreasing by y-coordinate value

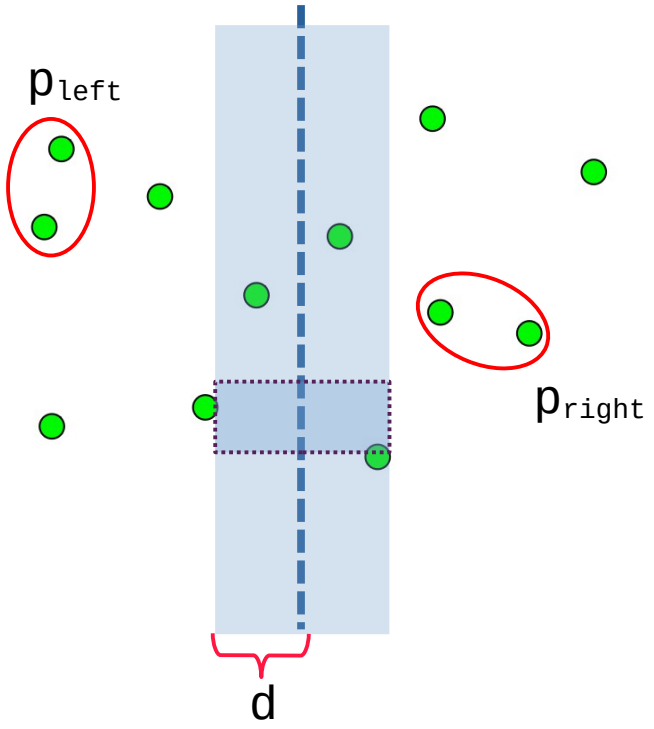
- It's possible that all points are there!



Region Near Split

For each point in S_y, we only need to check for neighbors in the d x 2d box above the point

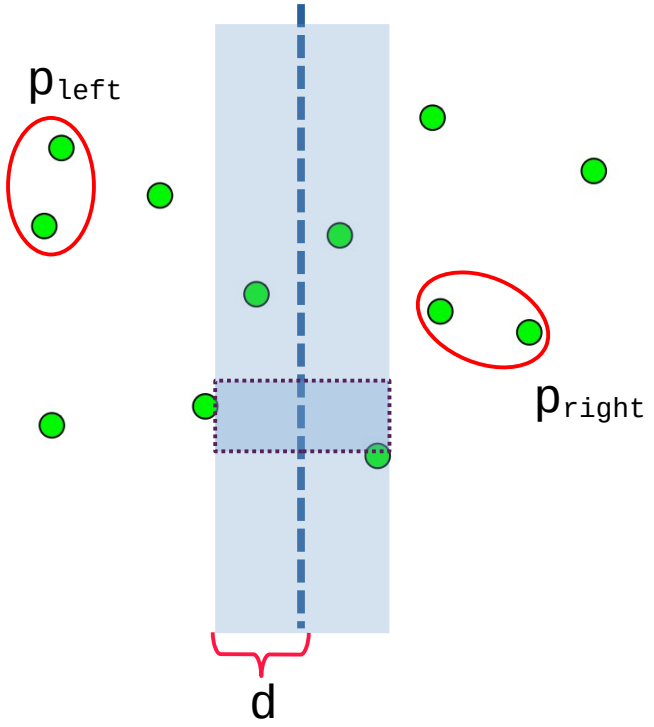
- How many point can fit in the box?



Region Near Split

For each point in S_y, we only need to check for neighbors in the d x 2d box above the point

- How many point can fit in the box?
- 8: at most, four on each side
- So, we only need to check the next 7 points in S_y



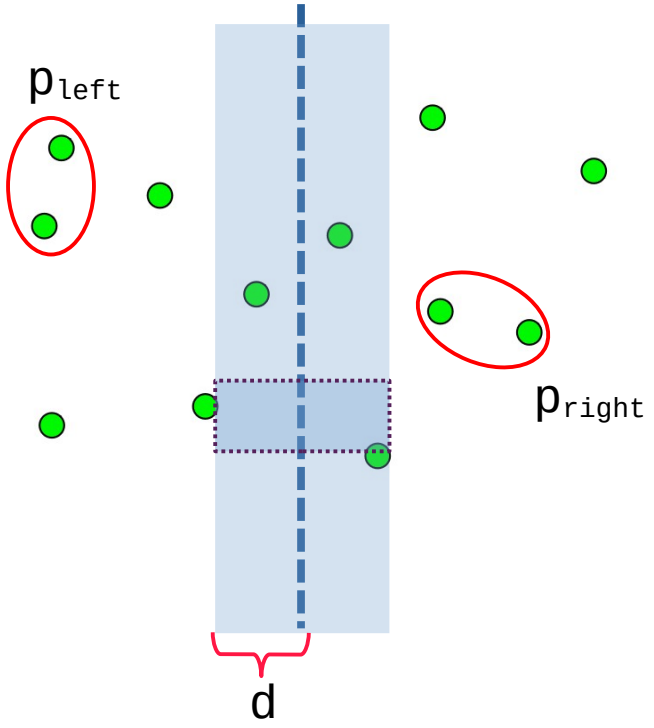
Region Near Split

For each point in S_y, we only need to check for neighbors in the d x 2d box above the point

- How many point can fit in the box?
- 8: at most, four on each side
- So, we only need to check the next 7 points in S_y
- Let d_{mid} be the minimum dist among S_y

Each check is constant time, and we need to check at most n points

- Running time?



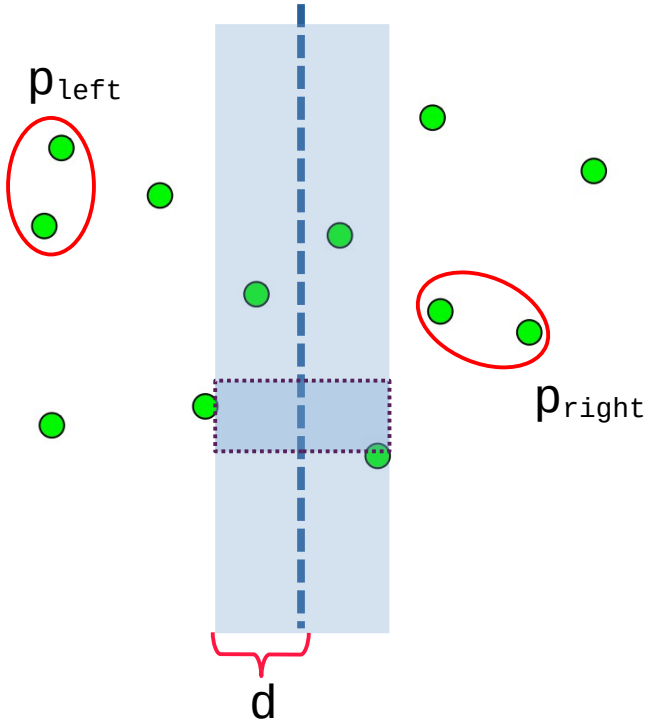
Region Near Split

For each point in S_y, we only need to check for neighbors in the d x 2d box above the point

- How many point can fit in the box?
- 8: at most, four on each side
- So, we only need to check the next 7 points in S_y
- Let d_{mid} be the minimum dist among S_y

Each check is constant time, and we need to check at most n points

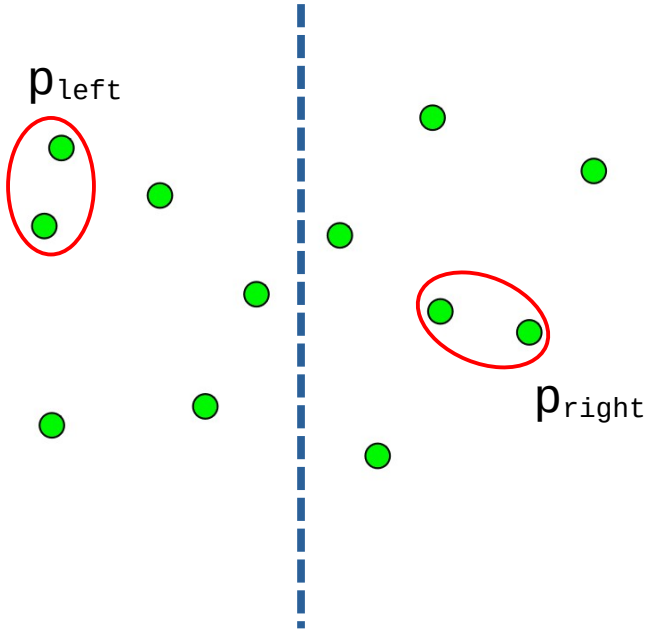
- Running time? O(n)



Combining

Let $d' = \min\{d_{left}, d_{right}, d_{mid}\}$

- Minimum distance over all points



Running Time

- Follows the same logic as Merge Sort
- Running time: `O(n log n)`

Next Time

- Efficient operations on trees